

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Case Study)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO4-AT2-CASE STUDY
Question	Q1 — Personal Finance Management Package in Python (50 Marks)
Section	Section D — Testing & Validation (10 Marks)

Q1. Section D — Testing & Validation [10 Marks]: Test the input validation and error handling of the Personal Finance Management package against deliberately bad data.

Test Strategy

Five distinct failure modes were identified from the requirements in Section A and the exception hierarchy in Section B: a negative amount, an unrecognised category, a malformed date, a reference to a non-existent account, and an expense that would overdraw an account. A dedicated CSV file containing the first four bad rows was run through `load_transactions_csv()` (which is built to collect errors rather than stop at the first one), and the fifth case (insufficient funds) was tested separately since it depends on account balance state rather than the row's own fields.

`bad_transactions.csv (deliberately invalid rows)`

```
date,type,category,amount,description,account
2026-07-09,Expense,Food,-500,Refund glitch,Bank
2026-07-11,Expense,Groceries,600,Wrong category name,Bank
2026-13-40,Expense,Food,500,Bad date,Bank
2026-07-13,Expense,Food,300,Unknown account,Wallet
```

Expected error per row

- **Row 1** (amount = -500): should raise **InvalidAmountError**, since amounts must be positive.
- **Row 2** (category = 'Groceries'): should raise **InvalidCategoryError**, since the recognised category is 'Food', not 'Groceries'.
- **Row 3** (date = '2026-13-40'): should raise **InvalidDateError**, since month 13 and day 40 do not exist.
- **Row 4** (account = 'Wallet'): should raise **InvalidAccountError**, since only 'Bank' and 'Cash' were registered.

`test_validation.py`

```
"""
test_validation.py
-----
Demonstrates validation of bad_transactions.csv: negative
amount, unrecognised category, malformed date, and unknown
account -- each caught as a distinct custom exception.
"""

from finance_manager import FinanceManager

manager = FinanceManager()
manager.add_account("Bank", opening_balance=20000)
manager.add_account("Cash", opening_balance=3000)
```

```

loaded, errors = manager.load_transactions_csv("bad_transactions.csv")
print(f"Loaded: {loaded}, Rejected: {len(errors)}")
for row, err in errors:
    print(f"  {row['date']} | {row['category']} | Rs.{row['amount']}: "
          f"{type(err).__name__} -> {err}")

print("\n--- Testing InsufficientFundsError (overdraft protection) ---")
try:
    manager.add_transaction("2026-07-31", "Expense", "Shopping", 100000,
                           "Big purchase", "Bank", allow_overdraft=False)
except Exception as e:
    print(f"Caught {type(e).__name__}: {e}")

```

Actual output of: python3 test_validation.py

```

Loaded: 0, Rejected: 4
  2026-07-09 | Food | Rs.-500: InvalidAmountError -> Transaction amount must be positive, got -500.0
  2026-07-11 | Groceries | Rs.600: InvalidCategoryError -> Unrecognised category 'Groceries'
  2026-13-40 | Food | Rs.500: InvalidDateError -> Invalid date '2026-13-40', expected YYYY-MM-DD
  2026-07-13 | Food | Rs.300: InvalidAccountError -> Unknown account 'Wallet'

--- Testing InsufficientFundsError (overdraft protection) ---
Caught InsufficientFundsError: Insufficient funds in 'Bank': balance 20000.00, attempted expense 100
000.00

```

Result vs Expectation

Row / Scenario	Expected Exception	Actual Exception	Result
Row 1 (amount -500)	InvalidAmountError	InvalidAmountError	PASS
Row 2 (category Groceries)	InvalidCategoryError	InvalidCategoryError	PASS
Row 3 (date 2026-13-40)	InvalidDateError	InvalidDateError	PASS
Row 4 (account Wallet)	InvalidAccountError	InvalidAccountError	PASS
Overdraft test (Rs.100000 vs Rs.20000)	InsufficientFundsError	InsufficientFundsError	PASS

Robustness check: partial-failure behaviour

A key requirement from Section A (robustness) was that one bad row must not stop an entire batch import. The test confirms this: `load_transactions_csv()` processed all 4 bad rows, correctly rejected every single one with its specific exception type, and returned (`loaded=0, errors=[...]`) rather than raising and aborting after the first bad row. In the earlier, successful run against `transactions_input.csv` (Section C), all 18 valid rows loaded with 0 rejections in the same code path, confirming that good and bad data are handled correctly by the same validation logic.

Conclusion

All five targeted failure modes were reproduced and correctly caught as their specific, documented exception type, and the batch import path was confirmed to isolate bad rows rather than abort the whole run. Combined with the 18/18 successful load in Section C, this confirms the validation layer is both strict on bad input and permissive on well-formed input, satisfying the data-integrity and robustness requirements set out in Section A.